



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • START THE CLUSTER, THEN GET A CLIENT • STEPS 1 AND 2

1 ☐ Create a container that is a Kubernetes cluster

```
mkdir kube
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

`-d` Detached. It runs in the background and you get your prompt back.
`--name k8s` A name of your choosing. Without it Docker still identifies the container, but by its id: 8d7726f96f747554440565a92a034cda1f767726619081595bab18cdf76987. Every later command would have to carry that, or its short form 8d7726f96f74.

`--privileged` Lifts the usual restrictions. k3s mounts cgroups and runs **its own** container runtime inside this container.
`--tmpfs /run` Two small writable in-memory filesystems k3s needs (also `/var/run`).
`-p 6443:6443` **HOST:CONTAINER.** Port 6443 on your laptop is forwarded to 6443 inside the container. 6443 is where the cluster's **API server** listens, and step 2 is going to talk to it.

`-v "$PWD/kube:/output"` Bind-mounts your new `kube` folder into the container, so a file written there lands on your laptop.
`rancher/k3s:v1.36.4-k3s1` The image. `rancher/k3s` is the repository, `v1.36.4-k3s1` the tag. You named no registry, so Docker went to **Docker Hub**.

`server` **Not a docker flag.** Everything after the image name is the command handed to the program inside. `server` tells k3s to be the cluster.

`--disable=traefik` Also k3s's. Traefik is an **ingress controller**: it routes HTTP arriving from outside to services inside. It claims ports 80 and 443 and adds startup time; nothing today arrives from outside.

`--write-kubeconfig` Writes the cluster's address and credentials to `/output/config`, which is your `kube/config`. `-mode 644` makes it readable without `sudo`.

After a few seconds `kube/config` exists on your laptop. Open it. One line reads `server: https://127.0.0.1:6443` — that is the only address anything will need.

2 ☐ Get a kubectl, and point it at that address

`kubectl` is **not part of the cluster**. It is an ordinary program that sends HTTPS requests to the address above, so it can live anywhere. Pick one.

(a) On Linux — install it on your laptop

```
sudo snap install kubectl --classic
export KUBECONFIG=$PWD/kube/config
```

It reaches the cluster through the port you published in step 1. This is exactly how you would drive a real cluster from your own machine.

(b) Anywhere, including macOS and Windows — run kubectl in its own container

```
docker run --rm -it --network container:k8s \
  -v "$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config \
  --entrypoint sh alpine/kubectl
```

`--rm` Delete this container the moment you leave it. The cluster in step 1 used `-d` because it must outlive your prompt; this one is disposable and you can start another whenever you like.

`-it` `-i` keeps the input open, `-t` gives a terminal. Together: an interactive shell instead of a single command.
`--network container:k8s` Do not give this container its own network — **join the one the cluster container already has**. That is why `127.0.0.1:6443` works in here: it is the cluster's own loopback, which is exactly the address written in `kube/config`.

`-v "$PWD/kube:/kube:ro"` Bind-mount again, the other way round: your `kube` folder appears inside this container at `/kube`. `:ro` makes it **read-only** — a client has no business editing the cluster's credentials.

```
-e KUBECONFIG=/kube/config  Sets an environment variable inside the container. kubectl reads KUBECONFIG to find out which cluster to talk to. Note the path is the one inside this container, not the one on your laptop.  
--entrypoint sh             Overrides what the image runs by default. alpine/kubectl is built to run kubectl and nothing else, so without this you would get one command per docker run. Swapping in a shell lets you type many.  
alpine/kubectl              The image: kubectl and a small Alpine base, nothing else. No registry named, so Docker Hub again.
```

Type plain `kubectl` from here. Prove where you are: `command -v k3s` finds nothing, because there is no cluster in this container — only a client.

Either way, you are outside the cluster.

Nothing from here on is typed **inside** the cluster. That is the normal arrangement: the cluster is one thing, the client is another, and they meet over one HTTPS address. Which did you pick, (a) or (b)? _____

Now check that it reaches the cluster.

`kubectl get nodes` asks the API server for the machines that make up the cluster. There will be exactly one: the container you started in step 1. If this prints a table, your client found the cluster and everything after this will work.

```
kubectl get nodes
```

Expect something close to:

NAME	STATUS	ROLES	AGE	VERSION
7c19eb7874a1	Ready	control-plane	11s	v1.36.4+k3s1

Instead got `The connection to the server localhost:8080 was refused?` Then `kubectl` never found the config and fell back to a built-in default. Check that `kube/config` exists and is not empty, and that you are running from the folder that contains `kube`.

2 • SOME DEFINITIONS

Step 1 produced a Kubernetes cluster inside one container. Step 2 produced a `kubectl` outside it that can reach the cluster's API. These are the words the next four steps use.

- kubectl** The cluster's command-line client. It runs nothing itself: each command becomes an HTTPS request to `https://127.0.0.1:6443`, and the reply is printed. The same split as the `docker` command and the Docker daemon — one is typed, the other does the work.
- pod** **A running container, as Kubernetes counts it.** `docker run` produces a container; Kubernetes produces a pod, with the container inside it. A pod may hold two or three containers that must share a machine and a single IP address; here, each holds one. **Pods are never created directly.**
- Deployment** **A request, in writing.** This one says: *keep three pods of ghcr.io/oesteban/whalesay running*. Submitting it starts nothing. It records what should be true.
- controller** **The program inside the cluster that makes the request true.** It reads the Deployment (three wanted), counts the pods that exist (two), starts one. A second later it counts again, and it never stops counting. Hence a deleted pod reappears: the pod was removed, the request was not.
- Job** **A different request: run this image once, until it finishes.** Its controller starts one pod, waits for a successful exit, and stops. Nothing is replaced.

One more, because it appears on screen: **CrashLoopBackOff**. The whale prints and exits, so the controller starts it again. The interval grows — 10s, then 20s, then 40s, doubling to a five-minute ceiling — and during the wait, this is the status shown. It does **not** mean the container crashed. It means *stopped again, waiting before the next attempt*.

3 • DRIVE THE CLUSTER • STEPS 3 TO 6

Type these with the `kubectl` you set up in step 2.

3 ☐ Ask the cluster to keep a whale running

```
kubectl create deployment whale --image=ghcr.io/oesteban/whalesay --replicas=3
```

A **Deployment** says: this many of these should exist, at all times. You do not say when to start anything.

Why is the registry spelled out here, when step 1 named none?

Because there are two copies of this image and they are **not the same thing**. Run these two *on your laptop*, not in the cluster shell:

```
docker buildx imagetools inspect ghcr.io/oesteban/whalesay  
docker buildx imagetools inspect oesteban/whalesay
```

The first prints a Manifests: list with Platform: linux/amd64 **and** Platform: linux/arm64. The second prints no list at all: it is a single image, linux/amd64 only.

An Apple Silicon Mac runs an **arm64** Linux VM, and k3s's containerd has no emulation to fall back on. The Docker Hub copy would give `exec format error` and the pod would never start. The GHCR copy was built for both by GitHub Actions on Friday.

On Linux or Windows, on Intel or AMD: the Docker Hub copy runs perfectly well. Check the manifests above, then try it alongside: `kubectl create deployment hub --image=oesteban/whalesay --replicas=1` Which architecture is your laptop?

4 ☐ Look at the pods the controller made for you

`kubectl get pods` — once straight away, once about a minute later.

This lists the **pods** the Deployment created on your behalf. Two columns matter: **STATUS**, what the pod is doing right now, and **RESTARTS**, how many times the container inside it has been started again.

Take both readings in the **first two minutes**. The retries slow down as they go (see *CrashLoopBackOff* in block 2), so two late readings can show the same number and tell you nothing.

5 ☐ Ask for the same image to run once, instead of forever

`kubectl create job whale --image=ghcr.io/oesteban/whalesay -- cowsay 'a cluster ran me'`

A **Job** says: run this to completion, once. Same image, same cluster, same command — only the kind of object is different.

6 ☐ Read what it printed

`kubectl logs job/whale`

`kubectl logs` prints whatever a container wrote to its output. You did not have to find the pod: naming the Job was enough.

4 • WHAT YOU STARTED

From `kubectl get nodes`:

NAME	_____	VERSION	_____
STATUS	_____	AGE when first Ready	_____

5 • THE TWO READINGS, AND THE REASON

First reading

A minute later

STATUS	_____	STATUS	_____
RESTARTS	_____	RESTARTS	_____

Run `docker ps` on your laptop: compare CONTAINER ID with the NAME in panel 4.

The container did not crash. It printed its whale and exited **successfully**. So why is Kubernetes starting it again?

The Job in step 5 used **the same image** and did not do this. What did you change, in one sentence?

Then read what Kubernetes actually claims.

`kubectl describe pod <one of them>` and find the block called **Last State**.

Exit Code _____ Reason _____

Now read the message beside **State: Waiting**. It contains the words `restarting failed container`.

Was it a failure? Reconcile those two lines, in one sentence.

6 • WHAT THE LOGS WILL TELL YOU

Logs are the first place you look when something is wrong, and they are not only for things that are wrong. Try all four, inside the cluster.

<code>kubectl logs deploy/whale</code>	There are three pods. Which one did it pick, and how do you know?
<code>kubectl logs <pod> --previous</code>	The container that already died . Why would that ever be the only copy left?
<code>kubectl logs -l app=whale --prefix</code>	All three at once. What does the prefix tell you?
<code>kubectl logs <pod> --timestamps</code>	Now you can see when . How far apart are the restarts?

A pod is gone and you want to know why. Which of those four do you reach for, and what would you have lost if you had waited an hour?

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

7 • NOW USE THE IMAGE YOU BUILT ON FRIDAY

Everything so far ran **someone else's** image. Repeat step 2 with your own, the one your GitHub Actions workflow published on Friday:

```
kubectl create deployment mine --image=ghcr.io/<your-handle>/whalesay
```

Your handle _____ Did it pull? _____

If it did not, `kubectl describe pod` will name the reason. Write it down, then fix it.

Nobody told the cluster where `ghcr.io` is, and nobody gave it a password. So: what **is** a registry, and what had to be true of your package for this to work at all?

On Friday you pushed to **GHCR**. When you type `docker pull alpine` you get an image from **Docker Hub**, and you never type a registry name. Where did that default come from, and what is the full name your `alpine` actually has?

8 • THE CONTROLLER'S OWN ACCOUNT

In panel 5 you read **Last State** from `kubectl describe pod`. The same output ends with an **Events** block. Read it, then widen to the whole cluster:

```
kubectl get events --sort-by=.lastTimestamp | tail -8
```

Nobody wrote this log. Who did, and what is it a record of — what you asked for, or what the cluster did about it?

9 • TWO IMAGE STORES, ONE NAME

Inside the cluster: `crictl images`.

On your laptop, in another terminal: `docker images`.

whalesay in `crictl`? _____ in `docker`? _____

Nobody copied anything from your laptop into the cluster. So how did the image get there?

10 • A WISH IS NOT A COMMAND

`kubectl scale deployment whale --replicas=5`, wait, then `kubectl scale deployment whale --replicas=0`. Count the pods after each.

pods at 5 _____ pods at 0 _____

Now delete the **Job's** pod: `kubectl delete pod <the job pod>`. Does it come back? Does a Deployment's pod come back? Why are the answers different?

11 • EXTRA MILE • SATISFY THE DEPLOYMENT INSTEAD OF ABANDONING IT

In step 5 you stopped the restarting by changing the **object**. There is a second way that leaves it a Deployment: same image, no rebuild, no re-push, sitting at `RESTARTS 0` and `Running`. Write the command.

You have now satisfied a Deployment two ways. Which would you use for a web server, and which for a nightly report?

No hint. The image runs one program and then stops; nothing says it has to.

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.